

- **Query patterns/use cases:** Application read and write patterns are a key decider because the data being stored is eventually for an application to read or write. So it's crucial to make it read and/or write friendly. As one delves into the data flow level details of any application design, the query patterns become clearer. For instance, an application may want to look up a particular value against a key, in which case a row/document oriented database would make more sense, or say, get the keys for a range of values, in which case a columnar database would be a better choice.
- **CAP theorem:** Production databases are generally deployed as distributed systems for the sake of fault tolerance and reliability. Hence various factors come into play, namely, consistency (i.e., how consistent is the value returned by all the members in a cluster), availability (i.e., how operational is the system despite failures or network partitions), and partition tolerance (i.e., how tolerant is the distributed system for network failures among the members in the cluster). The famous CAP theorem helps one disambiguate as to what is required and what can be traded off among consistency, availability and partition tolerance. Given network partitions are practically unavoidable, if your use case demands consistency then you may need to trade off availability or vice versa. For instance, in a simple 2-node database system (one master and one replica), if the use case demands consistency then the master will need to block the transactions against a certain key until the slave also gets updated for that key. However, during this window of master-replica synchronisation, any updates against the key will be blocked, thereby trading off the availability.
- **Scale:** As you get into the non-functional aspects of the system, scale requirements get crucial. This concerns forecasting the volume of data and requests to the database. A few systems scale well horizontally while others do so vertically, and in some use cases, certain keys can end up being heavily accessed.
- **Performance:** This involves assessing the performance requirements of your application. At some point of system design, you will understand which queries are read-heavy and which are write-heavy. The application response time decides the database transaction latency requirements.
- **Interoperability with ecosystem:** A database may never end up being called only from one or two services. Over time new requirements can add up, and your database technology should be able to communicate with the rest of your infrastructure with ease. For instance, as your organisation expands you may wish to export the data from your database to a warehouse or, say, trigger actions on row updates, at which point your database should integrate well with the adjacent technologies in your architecture. You can also look into the ecosystem support for the technology choice – in terms of how good the support is, open source knowledge and off-the-shelf available libraries (for example, ORM or object-relational mapping to manage the data access for relational databases).
- **Security and compliance:** In this age of growing fraud, security mechanisms like authentication, authorisation, encryption at rest and/or in transit are pretty basic offerings that you must look for. Compliance requirements can vary based on the organisation and can include handling sensitive data, ensuring the database complies with regulations, auditing, etc.
- **Development and operational overhead:** Eventually, it is the

developers who will maintain the system and hence need to ensure that their life is going to be easy while dealing with the decided choice. For instance, some databases are serverless while some need you to create and manage your own cluster. One also needs to evaluate how steep the learning curve will be for a newbie joining the team to learn about the technology.

- **Cost:** Cost is an important consideration when making the selection. Consider not only the upfront costs but also the long-term costs associated with scaling, maintenance, and support. Some databases are free to use while some are licensed.

Types of databases

Let's look into the broad types of databases based on data modelling that will usually be a pivot point for you to start making the choice.

Relational/row-based databases

- These databases store and organise data in the form of tables using rows and columns with a predefined schema. They are ideal for handling complex querying (e.g., joins between tables), transactional support (through commits/rollbacks), and data integrity (through constraints like primary/foreign key, etc) to support ACID properties. These are also called SQL-based databases, given SQL is the most common form of accessing data.
- A major downside is the schema extensibility and that makes these less suited for non-structural data or data whose structure is not known fully and can vary over time. This is where NoSQL (not only SQL) databases find their fit.
- Examples include MySQL, Postgres, and Oracle databases. These are widely used in finance/banking domains where data consistency/integrity is of paramount importance, and also ERP and CRM.